

BACHELOR'S THESIS COMPUTING SCIENCE

Evaluating and Validating the Information Extraction of a Music Newsletter LLM

Subtitle if you like

RUBEN KOELEWIJN
s1105300

June 3, 2026

First supervisor/assessor:
Title(s) Firstname Lastname

Second assessor:
Title(s) Firstname Lastname

Radboud University



Abstract

Brief outline of research questions, results. (The preferred size of an abstract is one paragraph or one page of text.)

Contents

1	Introduction	3
2	Preliminaries	5
2.1	Hallucination in Large Language Models	5
2.2	Cross-Lingual Natural Language Inference	6
2.3	Knowledge Graphs and Non-Parametric Memory	7
3	Cloudspeakers Concert Pipeline	8
3.1	Cloudspeakers Pipeline Architecture	8
3.2	Extracted Newsletters	9
3.3	Discussing Historical Feedback	10
4	Research	11
4.1	Pipeline Setup	11
4.1.1	Dataset and Corpus Construction	11
4.1.2	Composer	13
4.1.3	Dutch Parser	14
4.1.4	Validator	14
4.1.5	Classification	15
5	Results	16
6	Related Work	17
7	Conclusions	18
A	Appendix	21

Todo list

	Rework research question to better suit results/ conclusions	4
	Explain structure of the thesis.	4
	Might be nice to implement a piece on measuring succes.	7
	1. Moeten alle newsletter entries in de Appendix of is dat niet relevant?	10
	2. Moet deze in een footnote of als bron?	10
	Change Title	11
	Add table to show atomic claims give better results (maybe?)	13
	move footnote to introduction.	14
	find the studies that show LLM can verify themselves.	14
	add table with entailment Guidelines	14
	Psuedocode for to show the for loops and explain complexity?	15
	Back up claim that one sentence is better than full text.	15
	Classification could be an option	15

Chapter 1

Introduction

In recent years, large language models (LLMs) have emerged and evolved rapidly [14]. While initially restricted to research environments, these tools have now reached massive public scale. For instance, OpenAI recently confirmed it has surpassed 50 million paying subscribers across its commercial tiers, with up to 900 million weekly active users processing over 2 billion prompts per day [11]. This scale demonstrates that interacting with LLMs has become a routine global practice. However, while these models assist users in learning, working, and automating tasks, their widespread deployment does not come without downsides.

One primary risk is *hallucination*, a phenomenon, further discussed in Section 2, where an LLM generates output that is unfaithful to its original source or the user’s instructions. While minor discrepancies often go unnoticed, severe hallucinations can have profound consequences. For example, Zhao et al. [15] analyzed 2.5 million published papers and found a sharp rise in non-existent references following widespread LLM adoption, conservatively estimating over 146,000 hallucinated citations in 2025 alone. Similar impacts have been documented across other critical sectors. In healthcare, hallucinations pose severe ethical and safety risks [5], while in the commercial sector, the use of ungrounded conversational agents has led to concrete operational liabilities for enterprises [8].

As mentioned above, the occurrences of hallucinations are well-documented in high-stakes fields like medicine and law. In this thesis, however, we will consider a use-case which is less universally known, a case on a local music venue in Nijmegen. Since local venues often give the stage to smaller, lesser-known artists, the likelihood of hallucinations increases. While large language models demonstrate impressive factual recall for globally prominent topics, their reliability degrades significantly when processing niche or localized information. This limitation, often referred to as popularity bias, is formally quantified by Kandpal et al. [9], who demonstrate that an LLM’s ability to accurately state a fact is directly correlated to the frequency of that entity’s appearance in its pre-training corpus.

To address these widespread hallucinations, a common modern approach is the “LLM-as-a-judge” solution, wherein a highly capable model (such as GPT-4) is prompted to verify the factual accuracy of generated text. However, recent benchmarking reveals critical structural flaws in this methodology. Chen et al. [2] introduced the FELM benchmark, demonstrating that LLM evaluators struggle significantly to detect factual errors, often failing to identify hallucinations or confidently hallucinating false corrections themselves. This failure occurs because an LLM acting as a judge relies on the same flawed, popularity-biased parametric memory as the model generating the text, where a parametric memory refers to a model’s internal knowledge determined by the model’s internal knowledge. If an LLM lacks the training data to accurately describe

a local Dutch punk band, it also lacks the data required to fact-check a summary about that band. Consequently, relying on an LLM to evaluate long-tail cultural metadata creates a circular validation flaw.

To solve this, a new approach to automated evaluation is required. Traditional hallucination benchmarks often evaluate models against static, global datasets and primarily focus purely on English tasks. These traditional measures were not set up with testing small scale musical knowledge in mind, where the source text differs from the summarisation.

Therefore, we propose a two-phase hybrid evaluation architecture designed specifically for this domain.

The intuition behind this solution is to decouple the evaluation of reading comprehension from the evaluation of world knowledge. First, to assess whether the LLM faithfully translated and summarized the provided text, we utilize a cross-lingual Natural Language Inference (NLI) model (XLM-RoBERTa). This isolates intrinsic hallucinations. Second, to overcome the LLM's popularity bias regarding long-tail entities, we integrate a non-parametric, structured external database (MusicBrainz). By deterministically cross-referencing the LLM's claims against this real-world knowledge graph, we can catch extrinsic hallucinations that the NLI model alone would miss. Using manually labelled claims, we compare the accuracy of our built pipeline and are able to identify wide range of hallucinations types.

Our primary research question entails:

RQ 1 To what extent can a hybrid evaluation framework, integrating cross-lingual Natural Language Inference (NLI) with a structured external knowledge graph, effectively detect and classify intrinsic and extrinsic hallucinations in LLM-generated live music summaries?

RQ 1.1 How accurately can a cross-lingual NLI model isolate reading comprehension errors (intrinsic hallucinations) by evaluating the semantic entailment between English LLM summaries and their Dutch source texts?

RQ 1.2 How effectively can a non-parametric, external knowledge graph mitigate LLM popularity bias by verifying the real-world metadata of long-tail artists?

RQ 1.3 How can conflicting signals between intrinsic reading comprehension and extrinsic factual truth be formalized into a standardized error taxonomy (e.g., categorizing factual injections versus source conflicts)?

Rework research question to better suit results/conclusions

Explain structure of the thesis.

Chapter 2

Preliminaries

In this chapter, we introduce the concepts behind this thesis. First, we discuss the definitions of hallucinations in large language models. Secondly, we explain the subject of cross-lingual natural language interference and its relevance to this thesis. Lastly, we discuss knowledge graphs and its usage of non-parametric memory.

2.1 Hallucination in Large Language Models

In this thesis, we will be discussing the occurrences of hallucinations in LLMs. LLMs are massive artificial neural networks pre-trained on vast amounts of text data, designed to understand and generate human language. They show strong understanding in natural language and solving complex tasks [14]. While these outputs often contain useful and correct information, they are prone to *hallucinating*. Hallucination of a LLM consists of the model generating content that is not faithful to its source. Multiple types of hallucination exist, and following the taxonomy proposed by Huang et al. [6], we can categorise these into two types:

- **Factuality Hallucinations**

Discrepancies between generated content and verifiable real-world facts. This includes *Fabrications* (made-up claims) and *Contradictions* (statements that directly oppose reality).

- **Faithfulness Hallucinations**

Divergence from the provided user input or context. This includes *Instruction* (following the wrong instructions from the prompt), *Context* (Not understanding the context) and *Logical* (Incorrect logic, e.g. math errors). In information extraction (IE) tasks, these often manifest as “Missing Spans” or “Incorrect Types” [4].

Classifying the hallucinations allows us to have insight on what went wrong and where to improve, giving us the ability to do a system-level diagnosis of the pipeline [12]. For instance, a *faithfulness* error indicates a failure to maintain “context-boundary” stability [12], where a *factuality* error might need validation by means of an external source. Because LLMs are often unable to faithfully detect their own factual errors [2], deterministic verification is required to ensure airtight verification.

To understand why these factual hallucinations occur so frequently in localized domains like local music venues, we must look at how LLMs store information. LLMs encode knowledge within their network weights during pre-training, forming what is known as *parametric memory* [10]. While parametric memory may be reliable for globally prominent entities (e.g., world-famous

pop stars) due to their massive presence in the training data, it is inherently weak for niche, local entities. This limitation is known as *popularity bias*. Kandpal et al. [9] mathematically demonstrate that an LLM’s ability to accurately recall a fact drops immensely for “long-tail” entities that appear infrequently in its training corpus. Consequently, when an LLM is tasked with summarizing text about a local grassroots band, it encounters a void in its parametric memory and is prone to hallucinating plausible, fabrications in-line with what it knows about other grassroots bands.

2.2 Cross-Lingual Natural Language Inference

Natural Language Inference (NLI), also known as Recognizing Textual Entailment (RTE), is a core task in Natural Language Processing (NLP) [3]. NLP is a multidisciplinary field at the crossover between Computing Science, Linguistics, and Artificial Intelligence, where scientists focus on helping computers understand natural language. Common applications consist of machine translation, sentiment analysis, and automated summarisation. The latter is highly relevant for this thesis, since we deal with Information Extraction (IE), specifically extracting text and entities from websites [4], and Semantic Validation, which involves logically validating the semantic meaning of different sentences to detect errors [6, 2].

The task of NLI in NLP is determining the logical relationship between two pieces of text: a *premise* and a *hypothesis* [3]. Based on the truth of the premise, the model assigns one of three categorical labels as demonstrated in Table 2.1:

Table 2.1: Examples of Natural Language Inference (NLI) relationships.

Premise	Hypothesis	NLI Label
Frank Boeijen is from Nijmegen.	Nijmegen is the hometown of Frank Boeijen.	<i>Entailment</i>
Frank Boeijen is from Nijmegen.	Frank Boeijen is from Amsterdam.	<i>Contradiction</i>
Frank Boeijen is from Nijmegen.	Frank Boeijen is coming back to Nijmegen.	<i>Neutral</i>

- **Entailment:** The hypothesis is guaranteed to be true if the premise is true. In Table 2.1, the premise logically necessitates that Nijmegen is Boeijen’s hometown [3].
- **Contradiction:** The hypothesis is guaranteed to be false if the premise is true. Since the premise establishes Nijmegen as the origin, a claim of Amsterdam constitutes a direct contradiction [3].
- **Neutral:** The truth of the hypothesis cannot be determined from the premise alone. While Frank Boeijen may be “coming back” to Nijmegen, the original premise regarding his origin does not provide sufficient information to confirm or deny a return trip [3].

In the context of this thesis, we utilize *Cross-Lingual* NLI to bridge the language gap between Dutch source text and English generated summaries. By using a multilingual model like XLM-RoBERTa, we can evaluate a Dutch premise against an English hypothesis without an intermediate translation step, preserving the semantic nuances of the original concert metadata [3].

2.3 Knowledge Graphs and Non-Parametric Memory

To counteract the unreliability and popularity bias of an LLM’s internal parametric memory, modern NLP pipelines often integrate *non-parametric memory*-external, deterministically structured databases [10]. A primary form of non-parametric memory is a Knowledge Graph (KG).

A Knowledge Graph is an organized data structure that represents real-world entities as nodes (e.g., artists, venues, or albums) and the strict relationships between them as edges (e.g., “originates from” or “performs genre”). Unlike an LLM, which guesses facts based on statistical probability, a KG allows for absolute, deterministic data retrieval.

In order to replicate the pipeline structure with an integrated non-parametric for our niche local music knowledge, open-source databases such as MusicBrainz exist. The MusicBrainz database can be implemented as a knowledge graph, allowing us to cross-reference the generated claims against verified information. This option gives us the possibility to move on from “Neutral” to a possible “Contradiction” when the source text does not provide enough context.

Might be nice to implement a piece on measuring succes.

Chapter 3

Cloudspeakers Concert Pipeline

In this chapter, we will thoroughly discuss the architecture of the LLM summary we will be validation in this thesis, to give context on design choices made in later steps and chapters. The LLM we will be verifying was set up by Chris Bol [1].

3.1 Cloudspeakers Pipeline Architecture

Cloudspeakers is an Concert Event Pipeline setup by Chris Bol. It is an automated pipeline that collects concert events from Dutch music venues, extracts structured data using a Large Language Model, and distributes a weekly email newsletter to subscribers [1]. The system is focused on Utrecht-based venues such as ACU, Ekko, De Helling, db's, De Nijverheid, and Tivoli Vredenburg, but most importantly also contains the Nijmegen-based venue Doornroosje. The pipeline runs on Windmill, an open-source workflow automation platform. All logic is written in Python and SQL. No manual data entry is required.

The pipeline looks as follows:

1. Scrape event URLs from venue calendar pages
2. Store scraped pages (HTML + Markdown) in MinIO object storage
3. **LLM extracts structured event data (Deepseek)**
4. Structured data stored in PostgreSQL
5. Email newsletter to subscribers

Feedback is collected from users via a web form and stored in PostgreSQL.

Step 3, LLM extraction and summarisation by the Deepseek model, is what interest us the most. Even though we know the model and set up of the extraction, throughout this thesis the LLM and creation of the summary will be treated as a “black box”. The fields extracted by the LLM can be found in table 3.1.

Table 3.1: Description of extracted event metadata fields.

Field	Description
event	Name of the event
venue_name	Name of the venue (or hall)
main_act	Main performing artist or band
support_acts	Support acts (comma-separated)
date	Event date in YYYY-MM-DD format
date_sale	Ticket sale start date
url_sale	Ticket purchase URL
event_type	Category: gig, club night, open podium, talk, film, show, exhibition, other_category
genres	Music genres (comma-separated, lowercase)
summary	One-sentence English summary including artist origin
related_bands	Similar artists mentioned on the page

Table 3.1 shows that Deepseek is instructed to generate a one-sentence English summary including the origin of the artist, which will be used to as the to be validated summary for this thesis. Other relevant field include the `main_act` which is used to in generating atomic claims in Section 4.1.2 and `url_sale` which is used find the source text of the summary in Section 4.1.1. We limit our scope to verifying the `summary`, but investigative work shows that there exist occurrences of the LLM hallucinating on the `related_bands`, where all bands mentioned on the page get marked as related, even though this might not be the case. Other fields, will be discarded in our own validation steps.

3.2 Extracted Newsletters

For our tests, we will be using 36 different output examples, of events taking place on June 10 2026 untill April 15 2027. Listing 3.1 shows one example.

```

1  act: Frank Boeijen
2  date: 2026-08-23
3  url: https://doornroosje.nl/event/frank-boeijen-19/
4  venue: Openluchtteater De Goffert
5  bands: Frank Boeijen (main)
6  genres: pop
7  summary: Dutch singer-songwriter Frank Boeijen returns to his beloved
           hometown of Nijmegen for an evening of poetic songs and stories.
```

Listing 3.1: Example of a newsletter snippet.

Since current scope of the thesis does not include validating the `type`, `date`, `venue`, `genres`, we do not need to store these. The final newsletter is stored as a JSON object, allowing our pipeline easy access. Listing 3.2 shows a complete generated entry.

```

1 {
2   "event": "Daoud Live",
3   "venue_name": "Doornroosje",
4   "main_act": "Daoud",
5   "genres": ["jazz", "hip-hop"],
6   "summary": "French trumpeter Daoud is bringing his unique blend of jazz
7               and hip-hop to Nijmegen for the first time."
8 }

```

Listing 3.2: Example of a simplified newsletter entry.

All examples can be found in Appendix A.

3.3 Discussing Historical Feedback

At the bottom of each email sent by the pipeline, a link is put with a request for feedback to the reader. To see whether hallucinations already occurred during this pipeline, we requested the full feedback list from Bol.

The feedback list, contains 11 different feedback entries, of which three are suggestions of new features, two entries are about the **genre** being labelled incorrectly, two entries where the act (main vs supporting) is labelled wrong and lastly, two actual hallucinations.

The hallucination feedback reads: “I wonder whether *Luna Maki* really is from Utrecht.” A short online search shows that Luna Maki is actually from Amersfoort¹.

The second person notes: “*Naaz* genre not great. Text says ‘unique sound’ but the LLM turns it into regular folk music.” This claim was not easily confirmed, but still shows us that the LLM has problems with defining the origin and might be interpreting genres wrong.

1. Moeten alle newsletter entries in de Appendix of is dat niet relevant?

2. Moet deze in een footnote of als bron?

¹<https://www.nieuwsplein33.nl/nieuws/3867624/luna-maki-op-het-punt-van-doorbreken-met-afrobeat-uit-amersfoort>

Chapter 4

Research

Change
Title

This chapter, or series of chapters, delves into all technical details that are required to *prove* your scientific hypothesis. Please do not call this chapter or these chapters *Research* as that name is not descriptive. It should be sufficiently detailed and precise in order for any fellow computing scientist student to be able to *repeat* your research and therewith establish the same results / conclusions that you have obtained. Please note that, in order to improve readability of your thesis, you can put a part of this information also in one or more appendices (see Appendix A).

4.1 Pipeline Setup

In order to correctly verify the summary given by the LLM model, we have to create a pipeline that sources the needed context. As seen in Chapter 3, the newsletter contains a one-sentence summary. Before we can validate whether this is true or not, we need to extract all context. This is done by loading the URL using a headless browser, where we scrape the Dutch text. This then gets parsed and cleaned, and sent to the validator. Meanwhile, the summary gets chopped into atomic claims, who then can be validated. A diagram can be found in figure 4.1. Each paragraph will explain in depth the architectural choices that were made and how each part of the pipeline works.

4.1.1 Dataset and Corpus Construction

Before any validation or evaluation is possible, we need to load the source data given in the newsletters url part. This allows us to create a so-called *corpus* which contains all available data we have on the artist we are evaluating. This corpus is setup as a deterministic and sandboxed evaluation dataset, rather than executing live web scraping and external API queries during the NLI and KG evaluation phases. By doing all the data loading up front, we save time loading all data again when our pipeline crashes later on, or if we wish to run the experiment again.

This process collects the data in three simple steps: loading the local venue data, fetching the MusicBrainz facts, and saving everything to a JSON file.

Extracting Venue Data

As mentioned in Chapter 3, our primary data source is a generated newsletter, where we will be focussing on the venue Doornroosje, located in Nijmegen. This is setup using an automated

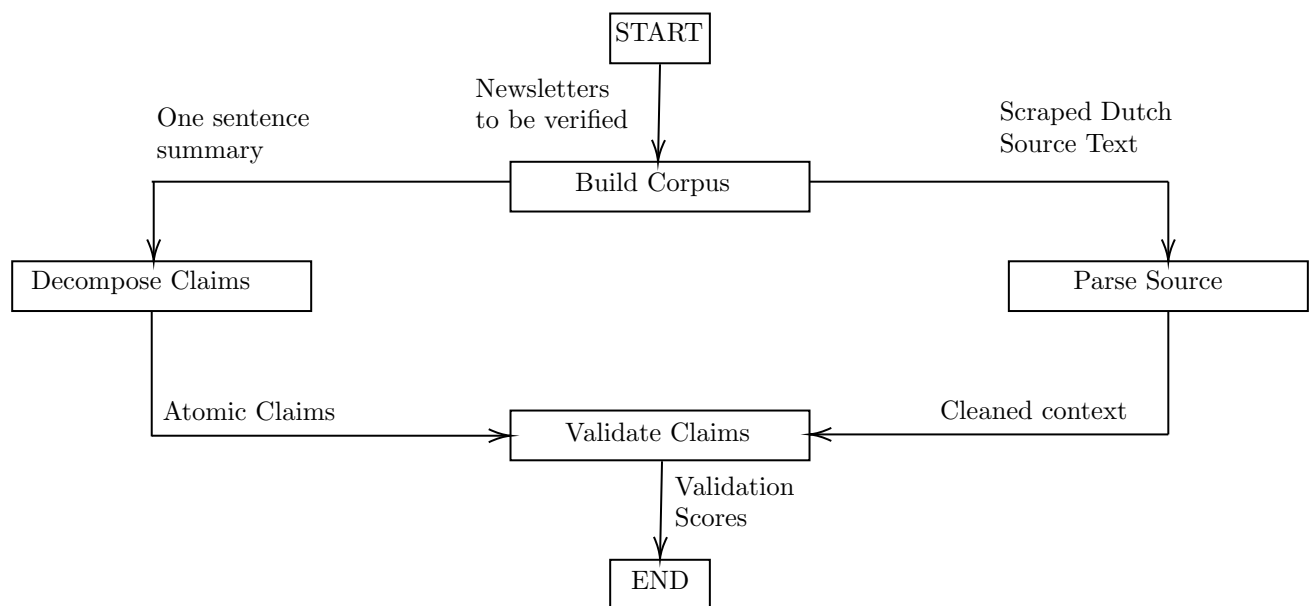


Figure 4.1: System architecture flowchart demonstrating the corpus ingestion, atomic claim decomposition branches, and cross-lingual source verification steps.

web scraper, where the primary event details and the LLM-generated summaries are manually extracted from these newsletters and provided to the script.

A comprehensive breakdown of the newsletter generation process, including its structural and visual layout, is discussed later in Chapter ?? (The Cloudspeakers Pipeline). For the purpose of constructing this evaluation corpus, the script strictly isolates the English promotional text—which serves as our target for hallucination detection—alongside the core billing metadata required to identify the performing artists.

Gathering MusicBrainz Knowledge

Once the primary event data is manually set up, the script automatically fills the dataset with verified real-world facts by querying the MusicBrainz API. To respect the strict rate-limiting policies enforced by the MusicBrainz infrastructure, the script utilizes a mandatory one-second delay between consecutive requests, ensuring the outbound traffic remains at or below *1request/second*.

For every artist identified in the venue data, the script searches the API to retrieve three specific data points: their geographic origin, associated musical genres, and any registered aliases. This factual data is stored to serve as the ground truth for detecting extrinsic hallucinations later in the pipeline.

Given that mid-tier venues frequently program niche, emerging, or highly localized artists, it is expected that some acts will be entirely absent from the MusicBrainz database. To ensure the robustness of the dataset generation, the script implements explicit error handling. If an artist cannot be resolved, the pipeline catches the exception and assigns empty values to the factual fields rather than terminating the program, allowing the script to safely proceed to the next entity.

4.1.2 Decomposer

The main task of the decomposer step is to view the claims made by the LLM in the generated summary. A summary can contain from one until multiple claims, so it is important that all of them get separated into atomic claims and get evaluated. We define a claim as ‘atomic’ if it contains exactly one predicate-argument relationship, which allows the subsequent NLI model to render a definitive Entailment or Contradiction label without semantic noise.

While state-of-the-art frameworks like Google DeepMind’s SAFE [13] rely on LLM prompting to decompose long-form summaries into atomic claim, introducing yet another machine learning model and creating high operational and computational costs, we made the choice to do this in a deterministic fashion. Our methodology adopts the core multi-stage atomic decomposition paradigm pioneered by SAFE, but replaces the LLM parser with a deterministic, rule-based linguistic dependency parse utilizing `spaCy`. Splitting claims into atomic claims, has been proven to improve performance in NLI for multiple models [7] and our own small scale testing (See Table ??). Our decomposer has been developed in parallel to the framework given by Huang et al.

Crucially, by utilizing a deterministic linguistic dependency parser rather than a secondary LLM for claim decomposition, we avoid the ‘recursive hallucination’ risk. When an LLM is used to decompose a summary, it may introduce its own hallucinations during the parsing phase. Our rule-based approach ensures that the decomposition remains faithful to the text’s syntactic structure, effectively providing a stable, verifiable input for the subsequent NLI evaluation.

The core of our decomposer consists of treating a sentence as a hierarchical Dependency Tree, where words are nodes connected by functional relationships. These relationships mimic the function of words in English sentences, such as a *preposition*, *subject*, *object*. The enumeration below shows the implementation of the algorithm.

1. Convert the raw text into a machine-readable dependency tree. This allows the system to identify the grammatical roles of words, such as which terms function as the subject and which function as the object of an action.
2. Iterate through every token in the document to locate “action anchors.” The system prioritizes VERB and AUX (auxiliary verbs such as ‘is’ or ‘has’), as these words define the core events or states necessary for a factual claim.
3. Resolving Components:
 - Subject: Identify the nominal subject (‘nsubj’). If the subject is a pronoun (e.g., “He”), replace it with the primary entity name to ensure the claim remains contextually independent and verifiable.
 - Verb Phrase: Collect all auxiliary verbs to preserve the original tense (e.g., “is performing”).
 - Object: Locate the grammatical receiver of the action (direct object, attribute, or complement).
 - Modifiers: Identify surrounding phrases that provide extra context, such as time, location, or manner.
4. Combine the subject, verb phrase, and object to construct a coherent, standalone sentence that represents the core fact.
5. If the sentence includes modifiers, “multiply” the base statement by each modifier to generate individual, independent claims. This step converts one complex sentence into multiple simple, verifiable statements.

Add table to show atomic claims give better results (maybe?)

6. Perform final data hygiene. Remove exact duplicate claims and discard any fragments that are under 5 words in length. This ensures the final output contains only high-quality, meaningful information.

4.1.3 Dutch Parser

4.1.4 Validator

Theory

Validating the claims is the core part of the operation we are doing. The validation is a process of cross-language NLI, comparing the original Dutch text to the English summaries that are given.

To avoid the inherent bias of self-validation—where an LLM acts as an uncritical judge of its own generated output ¹, the option of using a generative model for the entailment check was discarded. **Even though multiple studies have shown this to be effective, we wish to find a way to do this in a deterministic way due to LLMs having the nature of being unpredictable and prone to hallucination, as mentioned in section 2.1.** Comparing the premise and the hypothesis can be done in multiple ways, but for this thesis we have made the choice to make use of Facebook’s XLM-RoBERTa [3].

XLM-RoBERTa, is a Transformer-based language model trained on 2.5 Terabytes of filtered CommonCrawl data across 100 different languages. XLM-R learns a shared mathematical representation of human language, which results in it being able to convert multiple languages into a mapped mathematical vector space, meaning different languages can be compared. Conneau et al. show that XLM-R is very competitive with strong monolingual models on XNLI benchmarks, demonstrating that it is possible to have a single large model for all languages. [3] For the Cloudspeakers Experiment (section 3), this saves us steps from translating between Dutch and English, or limiting the experiments to pages that *do* have an English variant. As mentioned in 2.2, NLI looks at the entailment, contradiction and neutral scores. XLM-R outputs these when given a premise and hypothesis and using these we can determine likelihood of a hallucination by the LLM.

Guidelines to the entailment scores can be found in Table XX.

INSERT TABLE HERE

Implementation

The `validator` is implemented in Python utilizing PyTorch and the HuggingFace `transformers` library, using the specific `joeddav/xlm-roberta-large-xnli` model.

As mentioned in section XX, the pipeline delivers multiple different claims $\mathcal{C}$ and the original text from the source. Since XLM-R is set up to only check one premise against a single hypothesis, we will run the model for every claim and every sentence in the source material and find which sentence results in the highest entailment and which in the highest contradiction. We chose to run the experiments in this way, since it showed the best results in testing compared to supplying the model with the entire source text at once. For any given claim ($c \in \mathcal{C}$) and a set of source sentences ($\mathcal{S}$), the system iterates through $\mathcal{S}$, tokenizing the premise-hypothesis pairs with truncation enabled. The raw logits output by the model are passed through a Softmax function to calculate a probability distribution across the earlier mentions classes: **contradiction**, **neutral** and **entailment**. The script holds two trackers for each claim:

¹In the Netherlands, this specific conflict of interest is known as acting such as the company ‘WC-Eend’, referencing a famous advertising campaign in which the brand recommends its own product (Wij van Wc-Eend adviseren Wc-Eend).

move footnote to introduction.

find the studies that show LLM can verify themselves.

add table with entailment Guidelines

1. `max_entailment`: The sentence in $\mathcal{S}$ that returns the highest *entailment* probability
2. `max_contradiction`: The sentence in $\mathcal{S}$ that returns the highest *contradiction* probability.

Testing results can be found in Table XX. These tests showed that XLM-R struggles when the premise and hypothesis aren't of the same length, most likely due to too much information being fed and flooding it with information, leading to a more neutral score. These results are back up by SOURCE HERE!!!

4.1.5 Classification

Psuedocode
for to show
the for
loops and
explain
complex-
ity?

Back up
claim that
one sen-
tence is
better
than full
text.

Classification
could be
an option

Chapter 5

Results

This chapter will be about the results of the experiment.

Chapter 6

Related Work

In this chapter you demonstrate that you are sufficiently aware of the state-of-art knowledge of the problem domain that you have investigated as well as demonstrating that you have found a *new* solution / approach / method.

Chapter 7

Conclusions

In this chapter you present all conclusions that can be drawn from the preceding chapters. It should not introduce new experiments, theories, investigations, etc.: these should have been written down earlier in the thesis. Therefore, conclusions can be brief and to the point.

Bibliography

- [1] Chris Bol. Cloudspeakers - concert event pipeline. Internal technical documentation. Provided directly by the author., March 2026.
- [2] shiqi chen, Yiran Zhao, Jinghan Zhang, I-Chun Chern, Siyang Gao, Pengfei Liu, and Junxian He. Felm: Benchmarking factuality evaluation of large language models. In *Advances in Neural Information Processing Systems*, volume 36, pages 44502–44523. Curran Associates, Inc., 2023.
- [3] Alexis Conneau, Kartikay Khandelwal, Naman Goyal, Luke Zettlemoyer, and Veselin Stoyanov. Unsupervised cross-lingual representation learning at scale. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, 2020.
- [4] Jianhao Gao and et al. An empirical study on information extraction using large language models. *arXiv preprint arXiv:2305.14450*, 2024.
- [5] Oxford Bioethics Research Group. Through the looking glass: ethical considerations regarding LLM-induced hallucinations to medical questions. *PubMed Central (PMC)*, PMC13022756, 2025.
- [6] Lei Huang, Weijiang Yu, Weitao Ma, Weihong Zhong, Zhangyin Feng, Haotian Wang, Qianglong Chen, Weihua Peng, Xiaocheng Feng, Bing Qin, and Ting Liu. A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions. *ACM Trans. Inf. Syst.*, 43(2), 2025.
- [7] Minghui Huang. Atomic-snli: Fine-grained natural language inference through atomic fact decomposition, 2026.
- [8] LSEO Tech Governance Insights. AI hallucination liability: Business risk explained. LSEO Legal and Enterprise Strategy Hub, April 2026.
- [9] Nikhil Kandpal, Haikang Deng, Adam Roberts, Eric Wallace, and Colin Raffel. Large language models struggle to learn long-tail knowledge. In Andreas Krause, Emma Brunskill, Kyunghyun Cho, Barbara Engelhardt, Sivan Sabato, and Jonathan Scarlett, editors, *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pages 15696–15707. PMLR, 23–29 Jul 2023.
- [10] Alex Mallen, Akari Asai, Victor Zhong, Rajarshi Das, Daniel Khashabi, and Hannaneh Hajishirzi. When not to trust language models: Investigating effectiveness of parametric and non-parametric memories. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (ACL)*. Association for Computational Linguistics, 2023.

- [11] OpenAI. The state of enterprise AI: 2025 report. Technical report, OpenAI, December 2025. Accessed: 2026-05-21.
- [12] Vaishali Vinay. Failure modes in llm systems: A system-level taxonomy for reliable ai applications, 2025.
- [13] Jerry Wei, Yang Chen, Denny Huang, Denny Zhou, and Quoc V. Le. Long-form factuality in large language models. In *arXiv preprint arXiv:2403.18802*, 2024. Pioneered the multi-stage atomic claim decomposition and verification paradigm.
- [14] Wayne Xin Zhao, Kun Zhou, Junyi Li, Tianyi Tang, Xiaolei Wang, Yupeng Hou, Yingqian Min, Beichen Zhang, Junjie Zhang, Zican Dong, et al. A survey of large language models. *arXiv preprint arXiv:2303.18223*, 2023.
- [15] Zhenyue Zhao, Yihe Wang, Toby Stuart, Mathijs Vaan, Paul Ginsparg, and Yian Yin. Llm hallucinations in the wild: Large-scale evidence from non-existent citations, 05 2026.

Appendix A

Appendix

Appendices are *optional* chapters in which you cover additional material that is required to support your hypothesis, experiments, measurements, conclusions, etc. that would otherwise clutter the presentation of your research.